



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

LABORATORY EXERCISE 4

ENHANCING THE DASHBOARD WITH ADVANCED JAVASCRIPT & INTERACTIVE DATA VISUALIZATION

Learning Objectives

- By the end of this laboratory exercise, students should be able to:
 - Implement advanced DOM manipulation techniques using Vanilla JavaScript (ES6) to create dynamic and interactive user experiences.
 - Fetch and handle data from external sources (APIs) or simulate data retrieval using asynchronous JavaScript (Promises, async/await).
 - Create interactive data visualizations (charts, graphs) using a JavaScript charting library (e.g., Chart.js, Plotly.js) to present data meaningfully.
 - Implement client-side data filtering, sorting, and searching functionality to enhance the usability of the Dashboard.
 - Apply responsive design principles to ensure all interactive components adapt seamlessly to different screen sizes.
 - Manage application state on the client side to dynamically reflect user actions and data changes.
 - Connect the Dashboard to a simulated or real backend API to understand data flow in a web application.

Prerequisite student experiences and knowledge

Before starting this exercise, students should have:

- Completed Laboratory Exercise 3 (Building a Responsive Login & Dashboard System with Bootstrap 5 and JavaScript).
- Solid understanding of JavaScript ES6 syntax (arrow functions, template literals, destructuring, spread operator).
- Familiarity with the Document Object Model (DOM) and event handling in JavaScript.
- Understanding of asynchronous programming concepts (callbacks, Promises, async/await).
- Basic knowledge of JSON data format and how to work with it in JavaScript.
- Familiarity with browser developer tools (Chrome DevTools) for debugging.
- XAMPP and VS Code are properly installed and configured.
- A working GitHub repository from previous exercises.

Background

In Laboratory Exercise 3, you built a functional Login & Dashboard system with a responsive layout and basic interactivity. You simulated user authentication using localStorage and populated your Dashboard with static data. While this demonstrated fundamental JavaScript concepts and Bootstrap integration, real-world dashboards require more sophisticated functionality to be genuinely useful.

Advanced JavaScript and Interactive Data Visualization are what transform a simple dashboard into a powerful business intelligence tool. Modern dashboards must:

- Visualize Data Effectively: Raw numbers are hard to interpret. Charts, graphs, and visual indicators make complex data understandable at a glance.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

- Provide Interactivity: Users need to filter, sort, search, and drill down into data to uncover insights.
- Handle Real-Time Data: Dashboards often display live data from APIs or databases that update continuously.
- Be Responsive and User-Friendly: All interactive elements must work seamlessly across devices.

Your Theme Challenge (Extended):

For this exercise, you will continue with the business theme you selected in Laboratory Exercise 3. You are now tasked with enhancing your Inventory Management Dashboard by adding advanced features to make it a powerful decision-support tool. Your enhanced Dashboard must include:

1. **Interactive Data Visualizations:** Charts and graphs that display key metrics (e.g., inventory value by category, stock levels, sales trends).
2. **Dynamic Data Filtering:** The ability to filter inventory items by category, stock status (low stock, in stock, out of stock), or price range.
3. **Data Search Functionality:** A search bar that allows users to find specific products quickly.
4. **Real-Time Updates:** Simulated or actual real-time data updates that reflect changes in inventory status.
5. **Data Export:** An option to export inventory data to a CSV file for reporting purposes.
6. **Low Stock Alerts:** Visual alerts that notify users when items are running low.

Materials/Resources

- Personal Computer with Internet Access (Windows, macOS, or Linux).
- XAMPP Installer (downloaded from apachefriends.org).
- Visual Studio Code (downloaded from code.visualstudio.com).
- Web Browser (Google Chrome with DevTools is recommended).
- GitHub repository (from previous exercises).
- Chart.js (Recommended) chartjs.org or [Plotly.js](https://plotly.js/)
- Color Scheme Resources (for reference):
 - [Coolors.co](https://coolors.co/): Color palette generator
 - [Adobe Color](https://adobe.com/color/): Color wheel and theme creator

THEORY / CONCEPTS

Advanced DOM Manipulation with JavaScript

The Document Object Model (DOM) is the programming interface for HTML documents. In this exercise, you will use advanced DOM manipulation techniques to create dynamic, interactive content.

Key DOM Methods and Properties:

Method/Property	Description	Use Case
-----------------	-------------	----------



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

<code>document.querySelector()</code>	Selects the first element matching a CSS selector	Selecting specific elements for manipulation
<code>document.querySelectorAll()</code>	Selects all elements matching a CSS selector	Selecting groups of elements (e.g., all rows in a table)
<code>element.classList.add()</code>	Adds a CSS class to an element	Changing styles dynamically based on user actions
<code>element.classList.remove()</code>	Removes a CSS class from an element	Removing styles when no longer needed
<code>element.classList.toggle()</code>	Toggles a CSS class on and off	Creating toggle buttons or interactive states
<code>element.innerHTML</code>	Gets or sets the HTML content of an element	Dynamically updating content with new data
<code>element.textContent</code>	Gets or sets the text content of an element	Updating text without affecting HTML structure
<code>element.setAttribute()</code>	Sets an attribute on an element	Changing attributes like <code>src</code> , <code>href</code> , or custom data attributes
<code>document.createElement()</code>	Creates a new HTML element	Generating elements dynamically from data
<code>element.appendChild()</code>	Adds a child element to a parent	Building complex UI structures with JavaScript
<code>element.removeChild()</code>	Removes a child element	Removing elements when data is deleted

Asynchronous JavaScript and APIs

Many modern web applications use Asynchronous JavaScript to fetch data from servers without reloading the page. This is essential for creating smooth, responsive user experiences.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

Key Concepts:

- **Promises:** Represent the eventual completion (or failure) of an asynchronous operation. They can be in three states: pending, fulfilled (resolved), or rejected.
- **async/await:** A syntactic sugar built on Promises that makes asynchronous code look and behave more like synchronous code, improving readability.
- **Fetch API:** A modern interface for making HTTP requests in JavaScript. It returns Promises, making it ideal for fetching data from APIs.
- **JSON (JavaScript Object Notation):** A lightweight data interchange format that is easy for humans to read and write and easy for machines to parse and generate.

Example Data Flow:

1. User clicks a button to view inventory reports.
2. JavaScript makes a GET request to an API endpoint (or a local JSON file).
3. The API responds with data in JSON format.
4. JavaScript processes the data and updates the DOM dynamically.
5. The user sees the updated Dashboard without a page reload.

Data Visualization with JavaScript Libraries

Data visualization libraries simplify the process of creating complex charts and graphs. They provide pre-built components and configuration options that handle the heavy lifting of rendering visual elements.

Popular JavaScript Charting Libraries

LIBRARY	KEY FEATURES	BEST FOR
CHART.JS	Simple API, responsive, 8 chart types, HTML5 Canvas based	Quick, clean visualizations with minimal setup
D3.JS	Full control, data-driven approach, highly customizable	Complex, bespoke visualizations with unique requirements
PLOTLY.JS	Interactive charts, 3D visualizations, scientific plotting	Advanced scientific or engineering data visualization

Common Chart Types and Their user

CHART TYPE	USE CASE	EXAMPLE IN INVENTORY DASHBOARD
BAR CHART	Comparing categories or	Product quantities by category



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

	groups	
LINE CHART	Showing trends over time	Sales trends over months
PIE CHART	Showing proportions of a whole	Percentage of inventory value by category
DOUGHNUT CHART	Similar to pie chart with center space	Stock distribution (in stock, low stock, out of stock)
HORIZONTAL BAR CHART	Comparing categories with long labels	Products with highest inventory value
SCATTER PLOT	Showing relationships between variables	Price vs. quantity relationship

Client-Side State Management

State management is the practice of managing the application's data and UI state. In a dashboard, this means keeping track of:

- Which data is currently displayed
- What filters or search terms are active
- Which chart is currently visible
- User preferences (e.g., dark mode, chart type)

State Management Approaches

1. Local Storage: Use localStorage to persist data across sessions.
2. Global JavaScript Objects: Store state in a central object accessible to different functions.
3. URL Parameters: Use the URL to store filter or view settings (e.g., ?category=hardware).
4. Event-Driven Updates: Use events to trigger state changes and update the UI accordingly.

Laboratory Activity

Part 1: Planning Your Enhanced Dashboard Features

Step 1: Identify Key Dashboard Improvements

Based on your business theme, identify at least five interactive features to add to your Dashboard:

1. Interactive Chart: Visualize key data (e.g., inventory distribution by category, sales trends, stock status).
2. Data Filtering: Allow users to filter products by category, stock status, or price range.
3. Search Functionality: Implement a search bar to find specific products by name or SKU.
4. Low Stock Alerts: Highlight products that are at or below their reorder level.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

5. Export Functionality: Export the current inventory view to CSV.
6. Real-Time Data Simulation: Simulate inventory updates to mimic a real-time system.

Step 2: Design the Data Structure for Charts and Visualizations

Create a plan for how data will be organized for visualization:

- Inventory Value by Category: Group products by category and sum their **quantity * unit_price**.
- Stock Status Distribution: Count products in each stock status (in stock, low stock, out of stock).
- Top Products by Value: Sort products by **quantity * unit_price** and display the top 5.
- Recent Inventory Changes: Display the most recent inventory transactions.

Step 3: Plan Your JavaScript Organization

Consider how to organize your JavaScript code:

- Configuration: Store constants and configuration objects.
- State Management: Track the Dashboard's current state (filters, chart types, etc.).
- Data Management: Functions to fetch, process, and transform data.
- DOM Updates: Functions to update the UI based on the current state.
- Event Handlers: Functions that respond to user interactions.
- Visualization: Functions to render and update charts.

Part 2: Implementing the Enhanced Dashboard HTML Structure

Step 1: Extend the Dashboard Page

Add new sections to your **dashboard.html** page to accommodate the new features:

1. Charts Container: Add a section to display charts (e.g., a row with cards for different chart types).
2. Filter Controls: Add dropdown menus, buttons, or sliders for filtering data.
3. Search Bar: Add a search input field at the top of the inventory table.
4. Alert Section: Add a section to display low-stock alerts.
5. Export Button: Add a button to export data to CSV.

Step 2: Add Chart Canvas Elements

For Chart.js, you need **<canvas>** elements to render charts:

1. Add canvas elements with unique IDs for each chart you plan to create.
2. Ensure the canvas elements are properly sized and responsive.

Step 3: Organize the Layout

Use Bootstrap's grid system to organize the new elements:

1. Place charts in a row with appropriate column widths.
2. Position filter controls near the top of the page.
3. Place the search bar above the inventory table.
4. Ensure the layout is responsive across devices.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

Part 3: Creating the JavaScript Data Management System

Step 1: Create a Data Management Module

Create a new JavaScript file (e.g., **js/dataManager.js**) to handle all data operations:

Implement the Following Functions:

- **initializeData()**: Sets up the initial data (either from a JSON file, API, or hardcoded sample data).
- **getProducts()**: Returns the current product list.
- **getProductById(id)**: Returns a single product by its ID.
- **getProductsByCategory(category)**: Returns products filtered by category.
- **getLowStockProducts()**: Returns products that are at or below their reorder level.
- **getStockStatistics()**: Returns statistics about the inventory (total products, total value, low stock count, etc.).
- **getCategorySummary()**: Returns aggregated data by category (total value, total quantity, etc.).

Step 2: Implement Data Filtering and Search

Add filtering and search capabilities:

Filter Functions:

- **filterByCategory(category)**: Filter the product list by category.
- **filterByStockStatus(status)**: Filter by stock status (e.g., "in stock", "low stock", "out of stock").
- **filterByPriceRange(min, max)**: Filter by price range.
- **applyFilters()**: Apply all active filters to the product list.

Search Function

- **searchProducts(query)**: Search products by name or SKU.
- **updateSearchResults(query)**: Update the displayed products based on the search query.

Step 3: Implement Data Export

Add functionality to export data to CSV:

- **exportToCSV(Data)**: Convert the current product list to CSV format.
- **downloadCSV(csvContent, filename)**: Create a download link for the CSV file.

Part 4: Creating Interactive Data Visualizations with Chart.js

Step 1: Install and Configure Chart.js

1. Include Chart.js library in your HTML using CDN.
2. Create a configuration file or object for chart settings.

Step 2: Create the Inventory Value by Category Chart

1. Process the product data to group by category and calculate total value.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

2. Create a bar chart with categories on the x-axis and total value on the y-axis.
3. Apply your color scheme to the chart's colors.

Step 3: Create the Stock Status Distribution Chart

1. Count products in each stock status (in stock, low stock, out of stock).
2. Create a pie or doughnut chart showing the distribution.
3. Use appropriate colors for each status (e.g., green for in stock, yellow for low stock, red for out of stock).

Step 4: Make Charts Responsive

Based on your theme, create at least one additional chart:

- Top Products by Value: Horizontal bar chart showing the top 5 products by inventory value.
- Category Quantity Distribution: Bar chart showing total quantity by category.
- Supplier Distribution: Pie chart showing products by supplier.

Step 5: Create Additional Charts

- Use Chart.js's responsive option to ensure charts resize with their containers.
- Adjust font sizes and padding for smaller screens.

Part 5: Implementing Advanced Dashboard Interactivity

Step 1: Implement Filter Controls

1. Category Filter: Create a dropdown that lists all categories. When selected, filter the products and update the table and charts.
2. Stock Status Filter: Add buttons or a dropdown to filter by stock status.
3. Price Range Filter: Add min and max input fields for price filtering.

Step 2: Implement Search Functionality

1. Add an input field at the top of the inventory table.
2. Listen for input events to update the search results in real-time.
3. Highlight matching terms in the search results.

Step 3: Customize the JavaScript for Your Theme

1. Identify products that are at or below their reorder level.
2. Display a prominent alert banner on the Dashboard.
3. Highlight low stock products in the inventory table.

Step 4: Customize the JavaScript for Your Theme

1. Add a button to export the current inventory data to CSV.
2. When clicked, generate a CSV file and download it.

Step 5: Customize the JavaScript for Your Theme

1. Create a function that simulates inventory updates at regular intervals.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

2. Update the data, charts, and tables automatically.
3. Show notifications when inventory changes occur.

Part 6: Applying Custom CSS and Theming

Step 1: Extend Your Custom CSS

1. Add styles for new interactive elements (charts, filters, alerts).
2. Ensure all interactive elements match your color scheme.
3. Add transitions and animations for smooth interactions.

Step 2: Style Chart Colors

1. Configure your charts to use your chosen color scheme.
2. Apply consistent colors across all visualizations.

Step 3: Create Responsive Styles

1. Adjust chart sizes and layouts for mobile devices.
2. Ensure filters and controls are usable on small screens.

Part 7: Connecting the Dashboard to a Backend

Step 1: Create a Simple API (PHP)

1. If using XAMPP, create a PHP script that serves product data.
2. Implement endpoints to retrieve products, add products, update stock, etc.
3. Use JSON format for data exchange.

Step 2: Fetch Data Using JavaScript

1. Replace static data with data fetched from the API.
2. Use fetch or axios to make HTTP requests to your backend.
3. Handle loading states and errors gracefully.

Step 3: Implement Write Operations

1. Update products through the API (e.g., adjusting stock levels).
2. Reflect UI changes without page reloads.

Part 8: Testing and Verification

1. Testing Checklist:

- [/] All charts load correctly with data.
- [/] Charts update when data changes.
- [/] Category filter works correctly.
- [/] Stock status filter works correctly.
- [/] Price range filter works correctly.
- [/] Search functionality finds products by name and SKU.
- [/] Search results update in real-time.
- [/] Low stock alerts are displayed correctly.
- [/] Data export creates a valid CSV file.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

- ☐ Real-time simulation updates data correctly.
- ☐ All interactive elements are responsive.
- ☐ Charts resize correctly on mobile devices.
- ☐ Console shows no errors.
- ☐ Performance is acceptable with large datasets.

Part 5: Push to GitHub

1. Stage and commit all changes to your local repository.
2. Push to your GitHub repository with a meaningful commit message:
Example: "Added Laboratory Exercise 4: Enhanced Dashboard with Advanced JavaScript & Interactive Data Visualization"
3. Ensure all screenshots are included in your lab report.

Output / Results

Take a screenshot of the following outputs and paste them into your lab report document:

- Screenshot 1: VS Code showing your complete dashboard.html with new interactive elements.
- Screenshot 2: VS Code showing your dataManager.js code.
- Screenshot 3: VS Code showing your chart configuration code.
- Screenshot 4: VS Code showing your filter and search implementation.
- Screenshot 5: VS Code showing your custom CSS with interactive element styling.
- Screenshot 6: Web browser displaying the Dashboard with charts visible.
- Screenshot 7: Web browser showing filter controls in action.
- Screenshot 8: Web browser showing search functionality.
- Screenshot 9: Web browser showing low stock alerts.
- Screenshot 10: Web browser showing the exported CSV file.
- Screenshot 11: Web browser showing the responsive layout on mobile (use Chrome DevTools).
- Screenshot 12: GitHub repository showing the history of commits and the updated folder.

Submission Guidelines

- ✓ Name the file as: Lab4_YourStudentId_YourLastName.pdf.
- ✓ Submit the file to your instructor via the Learning Management System (LMS) provided before the start of the next laboratory session.
- ✓ Required Format: Include all screenshots from the Output/Results section in your PDF report.
- ✓ Questionnaire: Answer all questions in the Questions section below in your report.
- ✓ Conclusion: Write a brief conclusion summarizing what you learned in this laboratory exercise.

QUESTIONS:



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

1. In this exercise, you implemented data filtering and search functionality. Explain how you used DOM manipulation techniques to update the inventory table dynamically without reloading the page. Describe the process of creating, updating, and removing DOM elements based on filtered data. Provide specific examples of DOM methods you used and explain why they are appropriate for this use case.
 - I used DOM manipulation techniques to update the inventory table dynamically through JavaScript by the dataManager's in-memory state. The filter controls and search box uses event listener whenever it updates calling refreshInventoryView() but the original array of the products are not affected. renderInventoryTable uses document.createElement() to rebuild the table and rows. While DocumentFragment provides the main performance of improving workload since it reduces the DOM updates.
2. Explain the charting library you chose for this exercise (e.g., Chart.js, Plotly.js). Why did you select this library, and what are its key advantages for building an inventory dashboard? Discuss alternative charting libraries you considered and explain why you chose your selected library.
 - I selected Chart.js since its simple to use especially its API configuration-base making it easier to create three charts; A bar chart, doughnut chart, and lastly a horizontal bar chart. Chart.js also have a good feature via allowing the data to be changed without duplication by real-time updates. Plotly.js is one of the cdn that I wanted to try since its powerful but on the otherhand I only needed a simple chart. That's why I chose Chart.js for its simplicity and capabilities for my simple JavaScript project.
3. Compare the approach to storing and managing data in this exercise (using JavaScript objects or APIs) with the approach in Laboratory Exercise 3 (using localStorage). What are the limitations of localStorage for a real-world dashboard application? Discuss the advantages and disadvantages of each approach in terms of data persistence, security, scalability, and user experience.
 - In lab 3, login was using localStorage which saves the information in rememberedUsername. In lab 4, inventory data uses dataManager's in-memory state to load from PHP API, with JSON as a fallback. The limitation of localStorage for a real-world dashboard application is that its only limited to the browser and device where it is on. That is why API-backed data are being used since it allows multiple users to share sources through different devices and access information with the same information. Although localStorage is faster and useful for a simple client-side info login flag, it is however not good for a real-time inventory system. That is why API-backed system provides performance.



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

Output / Results

```
1 let simulationIntervalId = null;
2
3
4 document.addEventListener('DOMContentLoaded', async function () {
5     const isLoggedIn = localStorage.getItem('isLoggedIn');
6     if (isLoggedIn !== 'true') {
7         window.location.href = 'index.html';
8         return;
9     }
10
11     const username = localStorage.getItem('user') || 'User';
12     updateGreeting(username);
13
14     const userNameSpan = document.getElementById('userName');
15     if (userNameSpan) {
16         userNameSpan.textContent = username;
17     }
18
19     updateStatistics();
20     populateActivityTable();
21     setupLogout();
22
23     showInventoryLoading(true);
24     try {
25         await DataManager.initializeData();
26     } catch (err) {
27         console.error(err);
28         showInventoryLoading(false);
29         showToast('Could not load inventory data - check the console.');
```

```
30     return;
31 }
32 showInventoryLoading(false);
33
34 renderInventoryDashboard();
35 setupFilterControls();
36 setupSearch();
37 setupExport();
38 startRealTimeSimulation();
39 });
40
41
42 > function updateGreeting(username) { ...
60 }
61
62 > function updateStatistics() { ...
82 }
83
84 > function populateActivityTable() { ...
```



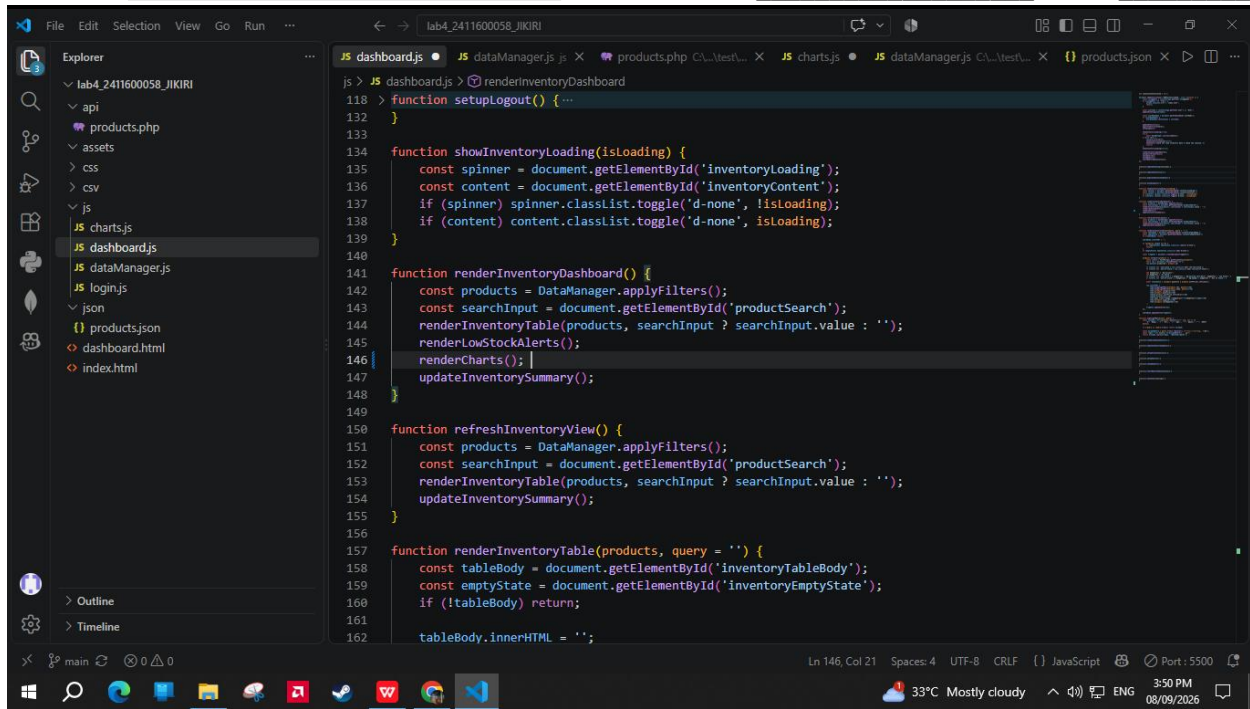
RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027

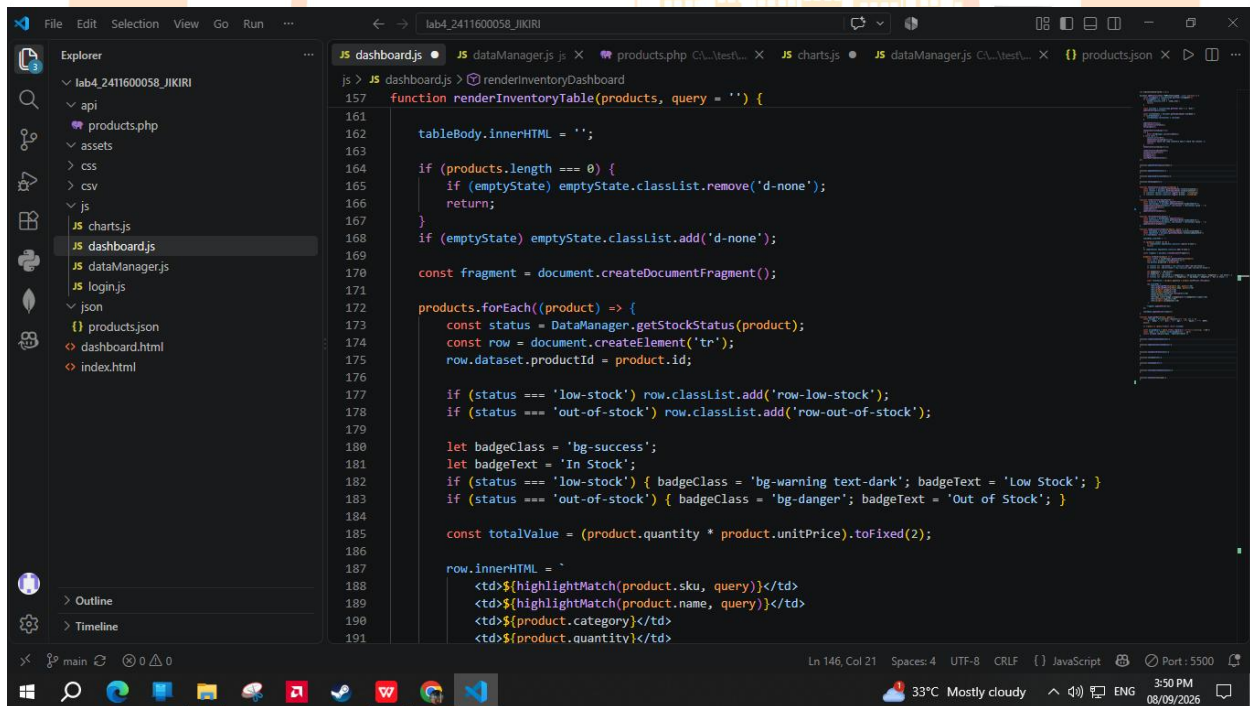


NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



The screenshot shows a Visual Studio Code editor window with the file explorer on the left displaying a project structure for 'lab4_2411600058_JIKIRI'. The main editor area shows the 'dashboard.js' file with the following JavaScript code:

```
js > JS dashboard.js > renderInventoryDashboard
118 > function setupLogout() { ...
132 }
133
134 function showInventoryLoading(isLoading) {
135     const spinner = document.getElementById('inventoryLoading');
136     const content = document.getElementById('inventoryContent');
137     if (spinner) spinner.classList.toggle('d-none', isLoading);
138     if (content) content.classList.toggle('d-none', isLoading);
139 }
140
141 function renderInventoryDashboard() {
142     const products = DataManager.applyFilters();
143     const searchInput = document.getElementById('productSearch');
144     renderInventoryTable(products, searchInput ? searchInput.value : '');
145     renderLowStockAlerts();
146     renderCharts();
147     updateInventorySummary();
148 }
149
150 function refreshInventoryView() {
151     const products = DataManager.applyFilters();
152     const searchInput = document.getElementById('productSearch');
153     renderInventoryTable(products, searchInput ? searchInput.value : '');
154     updateInventorySummary();
155 }
156
157 function renderInventoryTable(products, query = '') {
158     const tableBody = document.getElementById('inventoryTableBody');
159     const emptyState = document.getElementById('inventoryEmptyState');
160     if (!tableBody) return;
161     tableBody.innerHTML = '';
```



The screenshot shows a Visual Studio Code editor window with the file explorer on the left displaying a project structure for 'lab4_2411600058_JIKIRI'. The main editor area shows the 'dashboard.js' file with the following JavaScript code:

```
js > JS dashboard.js > renderInventoryDashboard
157 function renderInventoryTable(products, query = '') {
161     tableBody.innerHTML = '';
162
163     if (products.length === 0) {
164         if (emptyState) emptyState.classList.remove('d-none');
165         return;
166     }
167     if (emptyState) emptyState.classList.add('d-none');
168
169     const fragment = document.createDocumentFragment();
170
171     products.forEach((product) => {
172         const status = DataManager.getStockStatus(product);
173         const row = document.createElement('tr');
174         row.dataset.productId = product.id;
175
176         if (status === 'low-stock') row.classList.add('row-low-stock');
177         if (status === 'out-of-stock') row.classList.add('row-out-of-stock');
178
179         let badgeClass = 'bg-success';
180         let badgeText = 'In Stock';
181         if (status === 'low-stock') { badgeClass = 'bg-warning text-dark'; badgeText = 'Low Stock'; }
182         if (status === 'out-of-stock') { badgeClass = 'bg-danger'; badgeText = 'Out of Stock'; }
183
184         const totalValue = (product.quantity * product.unitPrice).toFixed(2);
185
186         row.innerHTML = `
187             <td>${highlightMatch(product.sku, query)}</td>
188             <td>${highlightMatch(product.name, query)}</td>
189             <td>${product.category}</td>
190             <td>${product.quantity}</td>
```



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

The image shows a VS Code editor window with a JavaScript file named `dashboard.js` open. The code defines a function `renderLowStockAlerts()` that checks for low stock products and displays an alert. The Explorer sidebar on the left shows the project structure, including folders like `api`, `assets`, `css`, `csv`, `js`, and `charts.js`, and files like `products.json`, `dashboard.html`, and `index.html`. The bottom status bar shows the file is at line 146, column 21, in UTF-8 encoding, with 4 spaces and CRLF line endings. The system tray at the bottom indicates the date is 08/09/2026 and the time is 3:51 PM.

```

215 }
216
217 function renderLowStockAlerts() {
218     const alertContainer = document.getElementById('lowStockAlert');
219     if (!alertContainer) return;
220
221     const lowStock = [...DataManager.getLowStockProducts()].sort((a, b) => a.quantity - b.quantity);
222
223     if (lowStock.length === 0) {
224         alertContainer.innerHTML = '';
225         return;
226     }
227
228     const items = lowStock
229         .slice(0, 6)
230         .map(p => {
231             const status = DataManager.getStockStatus(p);
232             const label = status === 'out-of-stock' ? 'OUT OF STOCK' : `${p.quantity} left (reorder at
233             return `<li><strong>${p.name}</strong> - ${label}</li>`;
234         })
235         .join('');
236
237     alertContainer.innerHTML = `
238     <div class="alert alert-warning alert-dismissible fade show d-flex" role="alert">
239         <i class="bi bi-exclamation-triangle-fill fs-4 me-3 flex-shrink-0"></i>
240         <div>
241             <strong>${lowStock.length} item(s) need attention</strong>
242             <ul class="mb-0 mt-2 small">${items}</ul>
243         </div>
244         <button type="button" class="btn-close ms-auto" data-bs-dismiss="alert" aria-label="Close"></button>
245     </div>
246 `;

```




RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
js > JS dashboard.js > renderInventoryDashboard
244   <button type="button" class="btn-close ms-auto" data-bs-dismiss="alert" aria-label="Close">
245   </div>
246   };
247 }
248
249 function updateInventorySummary() {
250   const stats = DataManager.getStockStatistics();
251   const setText = (id, text) => {
252     const el = document.getElementById(id);
253     if (el) el.textContent = text;
254   };
255   setText('invTotalProducts', stats.totalProducts);
256   setText('invTotalValue', `$$${stats.totalValue.toFixed(2)}`);
257   setText('invLowStock', stats.lowStockCount);
258   setText('invOutOfStock', stats.outOfStockCount);
259 }
260
261
262 > function setupFilterControls() { ...
314 }
315
316 > function setupSearch() { ...
328 }
329
330 > function setupExport() { ...
340 }
341
342
343 > function startRealTimeSimulation() { ...
355 }
356
357
```

```
js > JS dashboard.js > renderInventoryDashboard
249 function updateInventorySummary() {
250   setText('invOutOfStock', stats.outOfStockCount);
251 }
252
253
254
255
256
257
258
259
260
261
262 function setupFilterControls() {
263   const categorySelect = document.getElementById('categoryFilter');
264   const stockButtons = document.querySelectorAll('[data-stock-filter]');
265   const minPriceInput = document.getElementById('minPriceFilter');
266   const maxPriceInput = document.getElementById('maxPriceFilter');
267   const applyBtn = document.getElementById('applyFiltersBtn');
268   const resetBtn = document.getElementById('resetFiltersBtn');
269
270   if (categorySelect) {
271     const categories = [...new Set(DataManager.getProducts().map(p => p.category)).sort()];
272     categorySelect.innerHTML =
273       '<option value="all">All Categories</option>' +
274       categories.map(c => '<option value="$${c}">${c}</option>').join('');
275
276     categorySelect.addEventListener('change', () => {
277       DataManager.filterByCategory(categorySelect.value);
278       refreshInventoryView();
279     });
280   }
281
282   stockButtons.forEach(btn => {
283     btn.addEventListener('click', () => {
284       stockButtons.forEach(b => b.classList.remove('active'));
285       btn.classList.add('active');
286       DataManager.filterByStockStatus(btn.dataset.stockFilter);
287       refreshInventoryView();
288     });
289   });
290 }
```



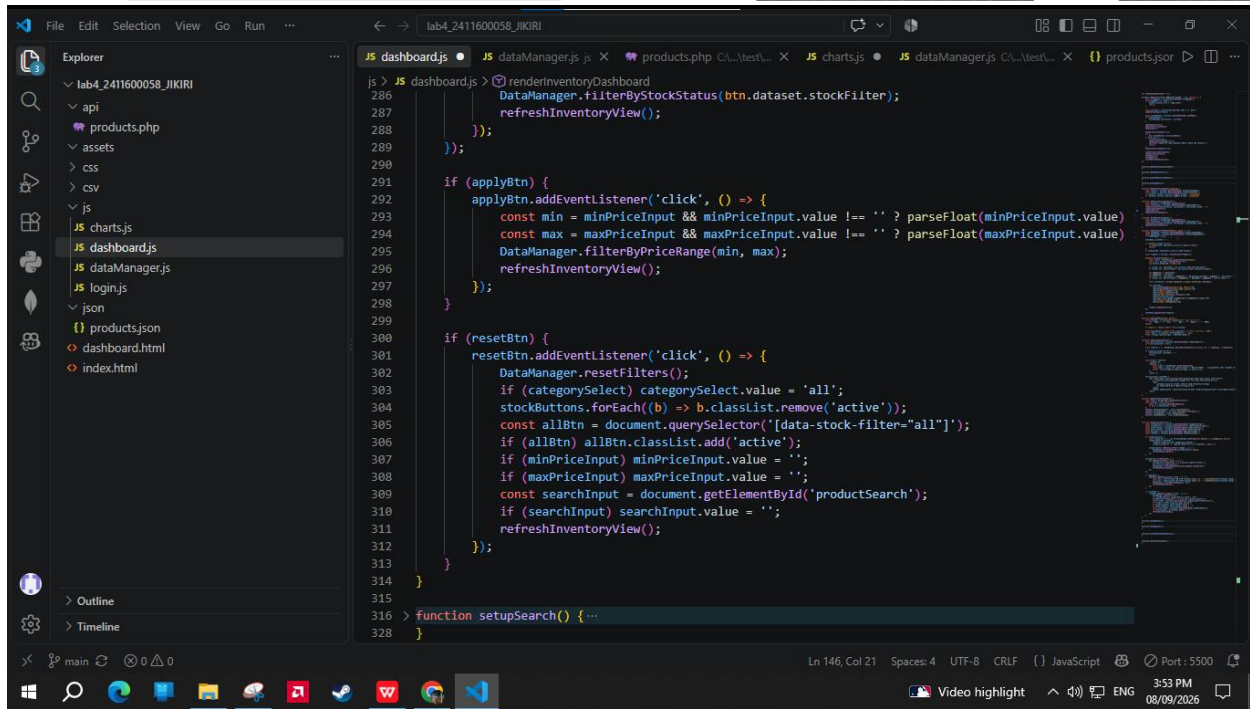
RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027

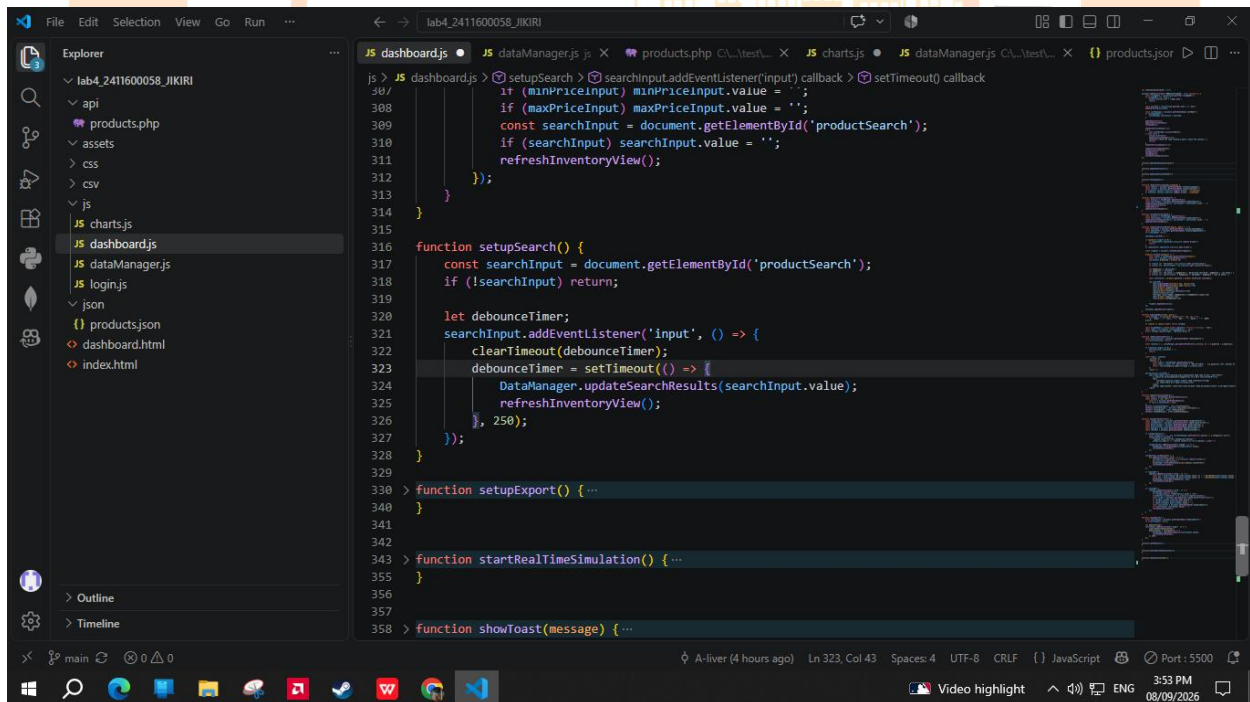


NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with folders like 'api', 'assets', 'css', 'csv', 'js', 'json', and 'products'. The code editor displays the 'dashboard.js' file, which contains JavaScript code for a dashboard. The code includes functions for rendering the inventory dashboard, filtering by stock status, and applying filters. The code is as follows:

```
js > JS dashboard.js > renderInventoryDashboard
286   DataManager.filterByStockStatus(btn.dataset.stockFilter);
287   refreshInventoryView();
288   });
289   });
290   });
291   if (applyBtn) {
292   applyBtn.addEventListener('click', () => {
293   const min = minPriceInput.value !== '' ? parseFloat(minPriceInput.value)
294   const max = maxPriceInput.value !== '' ? parseFloat(maxPriceInput.value)
295   DataManager.filterByPriceRange(min, max);
296   refreshInventoryView();
297   });
298   }
299   if (resetBtn) {
300   resetBtn.addEventListener('click', () => {
301   DataManager.resetFilters();
302   if (categorySelect) categorySelect.value = 'all';
303   stockButtons.forEach(b => b.classList.remove('active'));
304   const allBtn = document.querySelector('[data-stock-filter="all"]');
305   if (allBtn) allBtn.classList.add('active');
306   if (minPriceInput) minPriceInput.value = '';
307   if (maxPriceInput) maxPriceInput.value = '';
308   const searchInput = document.getElementById('productSearch');
309   if (searchInput) searchInput.value = '';
310   refreshInventoryView();
311   });
312   }
313   }
314   }
315   }
316   > function setupSearch() { ...
328   }
```



The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project structure with folders like 'api', 'assets', 'css', 'csv', 'js', 'json', and 'products'. The code editor displays the 'dashboard.js' file, which contains JavaScript code for a dashboard. The code includes functions for rendering the inventory dashboard, filtering by stock status, and applying filters. The code is as follows:

```
js > JS dashboard.js > setupSearch > searchInput.addEventListener('input') callback > setTimeout callback
308   if (maxPriceInput) maxPriceInput.value = '';
309   const searchInput = document.getElementById('productSearch');
310   if (searchInput) searchInput.value = '';
311   refreshInventoryView();
312   });
313   }
314   }
315   }
316   function setupSearch() {
317   const searchInput = document.getElementById('productSearch');
318   if (!searchInput) return;
319   let debounceTimer;
320   searchInput.addEventListener('input', () => {
321   clearTimeout(debounceTimer);
322   debounceTimer = setTimeout(() => {
323   DataManager.updateSearchResults(searchInput.value);
324   refreshInventoryView();
325   }, 250);
326   });
327   }
328   }
329   }
330   > function setupExport() { ...
340   }
341   }
342   }
343   > function startRealTimeSimulation() { ...
355   }
356   }
357   }
358   > function showToast(message) { ...
```



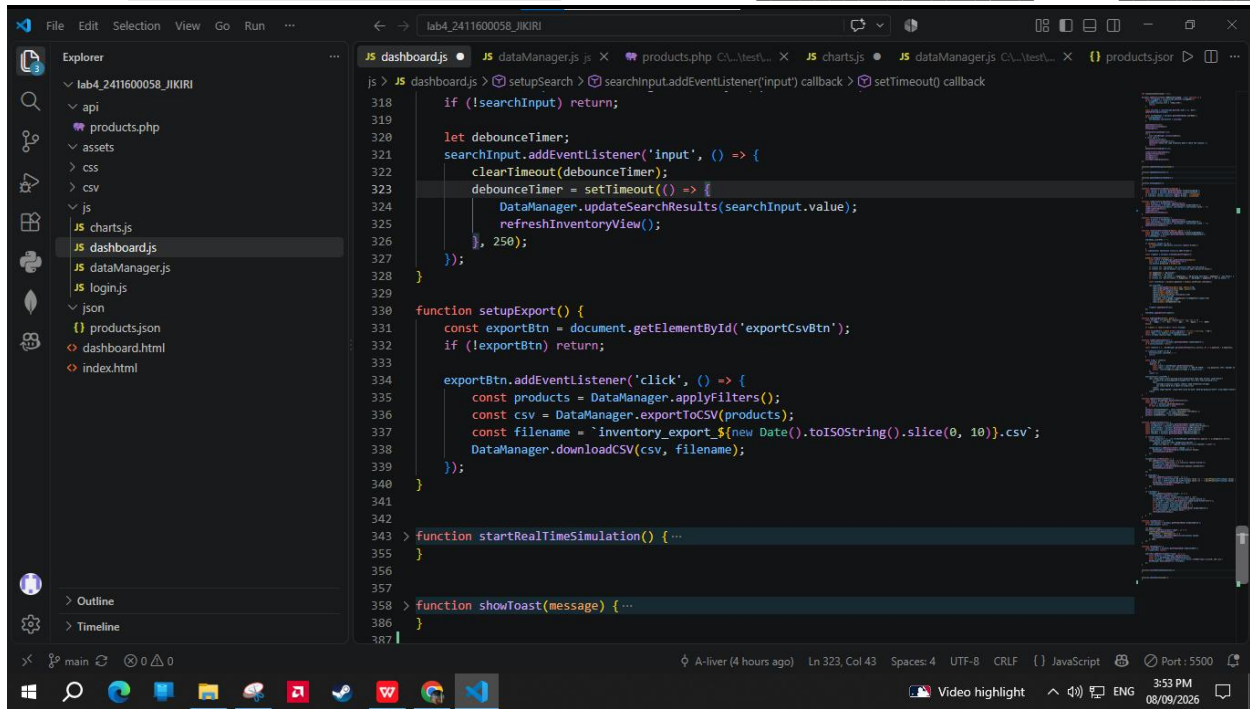
RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027

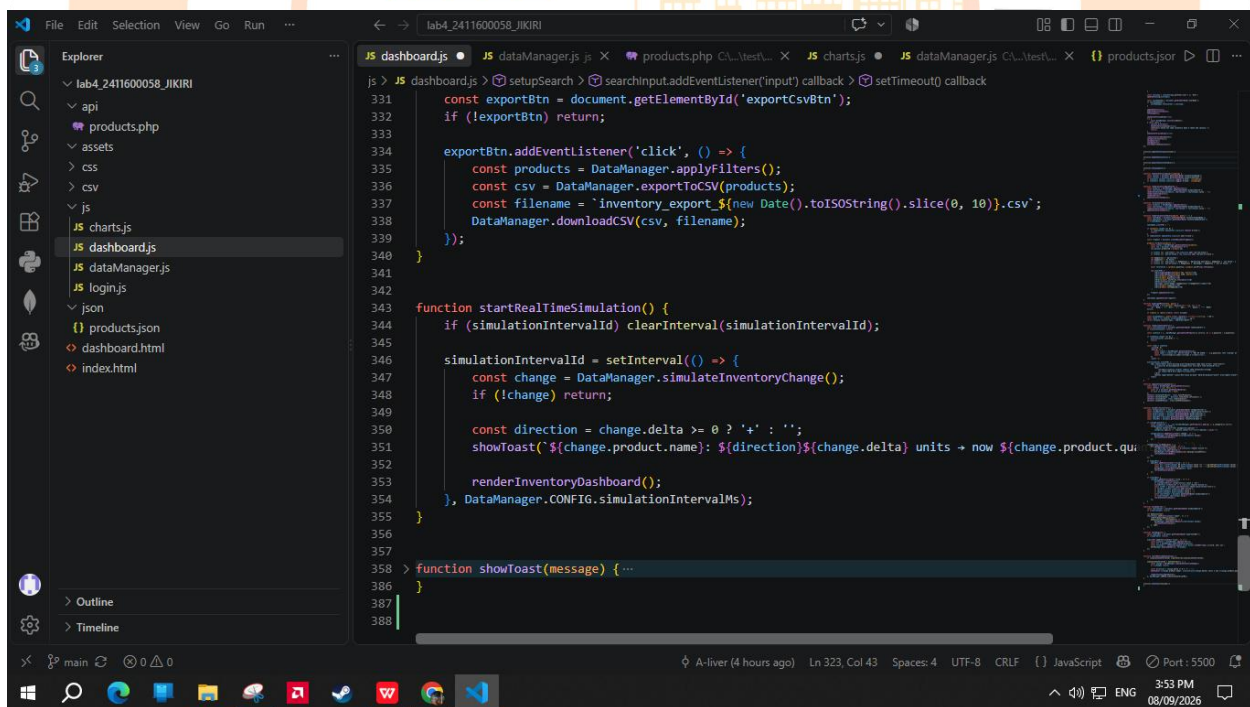


NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project structure for 'lab4_2411600058_JIKIRI' with files like 'products.php', 'charts.js', 'dashboard.js', 'dataManager.js', 'login.js', 'products.json', 'dashboard.html', and 'index.html'. The code editor displays the 'dashboard.js' file, which contains JavaScript code for a dashboard application. The code includes a search input event listener, a debounce timer, a function to update search results, a function to export data to CSV, and a function to start a real-time simulation. The code is as follows:

```
js > JS dashboard.js > JS dataManager.js > products.php C:\test\... > JS charts.js > JS dataManager.js C:\test\... > products.jsor > ...  
js > JS dashboard.js > setupSearch > searchInput.addEventListener('input') callback > setTimeout() callback  
318 if (!searchInput) return;  
319  
320 let debounceTimer;  
321 searchInput.addEventListener('input', () => {  
322   clearTimeout(debounceTimer);  
323   debounceTimer = setTimeout(() => {  
324     DataManager.updateSearchResults(searchInput.value);  
325     refreshInventoryView();  
326   }, 250);  
327 });  
328  
329  
330 function setupExport() {  
331   const exportBtn = document.getElementById('exportCsvBtn');  
332   if (!exportBtn) return;  
333  
334   exportBtn.addEventListener('click', () => {  
335     const products = DataManager.applyFilters();  
336     const csv = DataManager.exportToCSV(products);  
337     const filename = `inventory_export_${new Date().toISOString().slice(0, 10)}.csv`;  
338     DataManager.downloadCSV(csv, filename);  
339   });  
340 }  
341  
342  
343 > function startRealTimeSimulation() { ...  
344 }  
345  
346  
347  
348 > function showToast(message) { ...  
349 }  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500
```



The screenshot shows a Visual Studio Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project structure for 'lab4_2411600058_JIKIRI' with files like 'products.php', 'charts.js', 'dashboard.js', 'dataManager.js', 'login.js', 'products.json', 'dashboard.html', and 'index.html'. The code editor displays the 'dashboard.js' file, which contains JavaScript code for a dashboard application. The code includes a search input event listener, a debounce timer, a function to update search results, a function to export data to CSV, and a function to start a real-time simulation. The code is as follows:

```
js > JS dashboard.js > JS dataManager.js > products.php C:\test\... > JS charts.js > JS dataManager.js C:\test\... > products.jsor > ...  
js > JS dashboard.js > setupSearch > searchInput.addEventListener('input') callback > setTimeout() callback  
331 const exportBtn = document.getElementById('exportCsvBtn');  
332 if (!exportBtn) return;  
333  
334 exportBtn.addEventListener('click', () => {  
335   const products = DataManager.applyFilters();  
336   const csv = DataManager.exportToCSV(products);  
337   const filename = `inventory_export_${new Date().toISOString().slice(0, 10)}.csv`;  
338   DataManager.downloadCSV(csv, filename);  
339 });  
340  
341  
342  
343 function startRealTimeSimulation() {  
344   if (simulationIntervalId) clearInterval(simulationIntervalId);  
345  
346   simulationIntervalId = setInterval(() => {  
347     const change = DataManager.simulateInventoryChange();  
348     if (!change) return;  
349  
350     const direction = change.delta >= 0 ? '+' : '-';  
351     showToast(`${change.product.name}: ${direction}${change.delta} units + now ${change.product.quantity}`);  
352     renderInventoryDashboard();  
353   }, DataManager.CONFIG.simulationIntervalMs);  
354 }  
355  
356  
357  
358 > function showToast(message) { ...  
359 }  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500
```



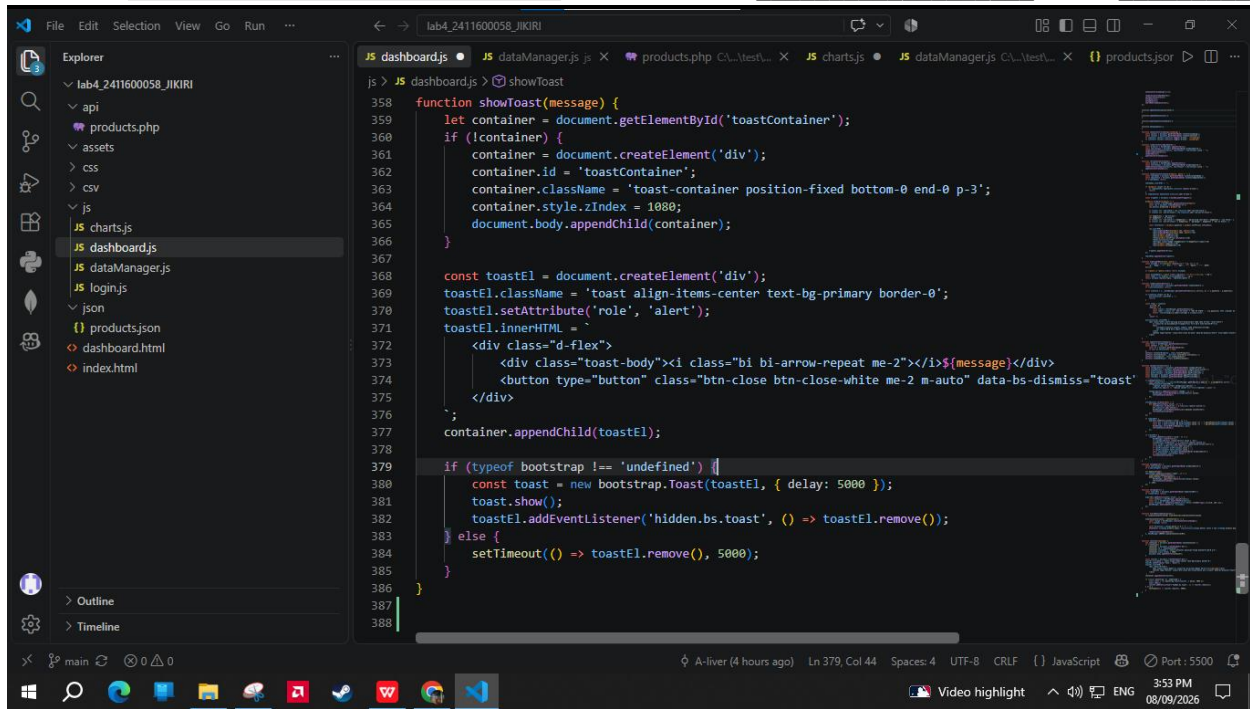

RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

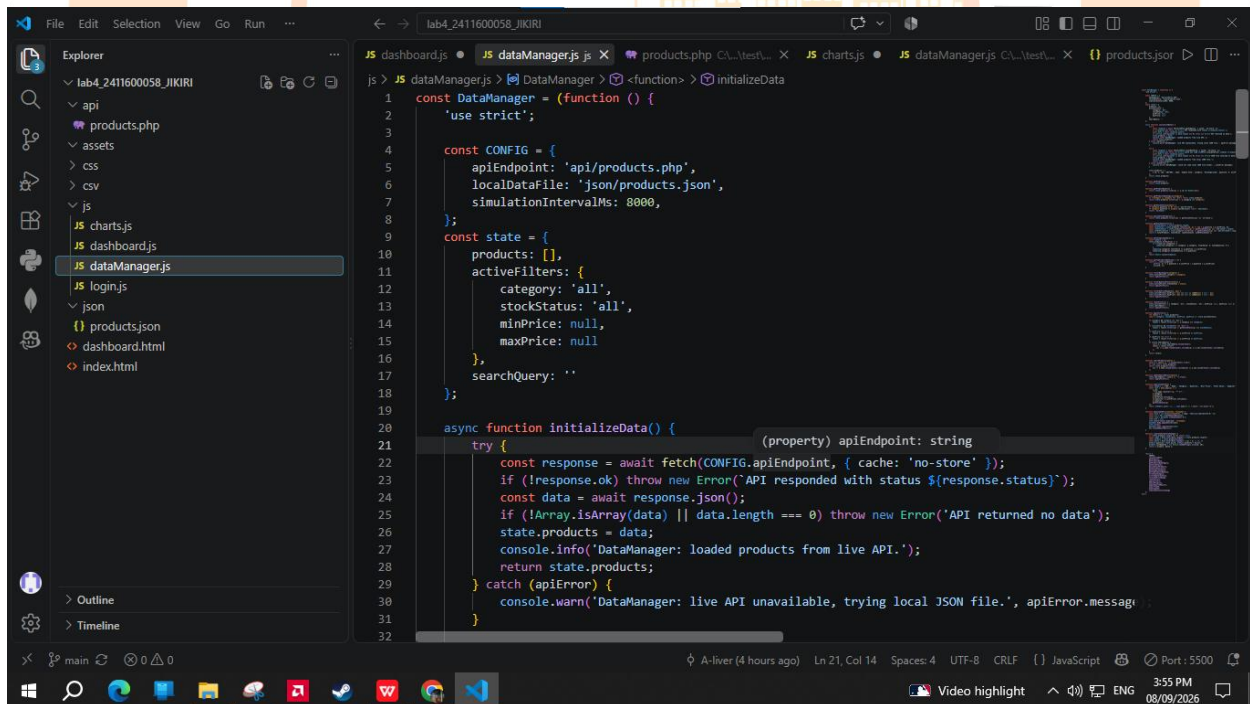
1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



```
js > JS dashboard.js > showToast
358 function showToast(message) {
359   let container = document.getElementById('toastContainer');
360   if (!container) {
361     container = document.createElement('div');
362     container.id = 'toastContainer';
363     container.className = 'toast-container position-fixed bottom-0 end-0 p-3';
364     container.style.zIndex = 1080;
365     document.body.appendChild(container);
366   }
367
368   const toastEl = document.createElement('div');
369   toastEl.className = 'toast align-items-center text-bg-primary border-0';
370   toastEl.setAttribute('role', 'alert');
371   toastEl.innerHTML = `
372     <div class="d-flex">
373       <div class="toast-body"><i class="bi bi-arrow-repeat me-2"></i>${message}</div>
374       <button type="button" class="btn-close btn-close-white me-2 m-auto" data-bs-dismiss="toast"></button>
375     </div>
376   `;
377   container.appendChild(toastEl);
378
379   if (typeof bootstrap !== 'undefined') {
380     const toast = new bootstrap.Toast(toastEl, { delay: 5000 });
381     toast.show();
382     toastEl.addEventListener('hidden.bs.toast', () => toastEl.remove());
383   } else {
384     setTimeout(() => toastEl.remove(), 5000);
385   }
386 }
387
388
```



```
js > JS dataManager.js > <function> > initializeData
1 const DataManager = (function () {
2   'use strict';
3
4   const CONFIG = {
5     apiEndpoint: 'api/products.php',
6     localDataFile: 'json/products.json',
7     simulationIntervalMs: 8000,
8   };
9
10  const state = {
11    products: [],
12    activeFilters: {
13      category: 'all',
14      stockStatus: 'all',
15      minPrice: null,
16      maxPrice: null
17    },
18    searchQuery: ''
19  };
20
21  async function initializeData() {
22    try {
23      const response = await fetch(CONFIG.apiEndpoint, { cache: 'no-store' });
24      if (!response.ok) throw new Error(`API responded with status ${response.status}`);
25      const data = await response.json();
26      if (!Array.isArray(data) || data.length === 0) throw new Error('API returned no data');
27      state.products = data;
28      console.info('DataManager: loaded products from live API.');
```



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
js > JS dataManager.js > DataManager > <function> > <function> > initializeData
1  const DataManager = (function () {
20     async function initializeData() {
32
33         try {
34             const response = await fetch(CONFIG.localDataFile, { cache: 'no-store' });
35             if (!response.ok) throw new Error('Could not load ${CONFIG.localDataFile} (status ${response.status})');
36             const data = await response.json();
37             if (!Array.isArray(data) || data.length === 0) throw new Error('JSON file returned no data');
38             state.products = data;
39             console.info('DataManager: loaded products from local JSON file.');
```

```
js > JS dataManager.js > DataManager > <function> > <function> > initializeData
1  const DataManager = (function () {
81
82     function getCategorySummary() {
83         const summary = {};
84         state.products.forEach((p) => {
85             if (!summary[p.category]) {
86                 summary[p.category] = { category: p.category, totalValue: 0, totalQuantity: 0 };
87             }
88             summary[p.category].totalValue += p.quantity * p.unitPrice;
89             summary[p.category].totalQuantity += p.quantity;
90         });
91         return Object.values(summary);
92     }
93
94     function getTopProductsByValue(n = 5) {
95         return [...state.products]
96             .sort((a, b) => b.quantity * b.unitPrice - a.quantity * a.unitPrice)
97             .slice(0, n);
98     }
99
100
101     function filterByCategory(category) {
102         state.activeFilters.category = category;
103         return applyFilters();
104     }
105
106     function filterByStockStatus(status) {
107         state.activeFilters.stockStatus = status;
108         return applyFilters();
109     }
110
111     function filterByPriceRange(min, max) {
```



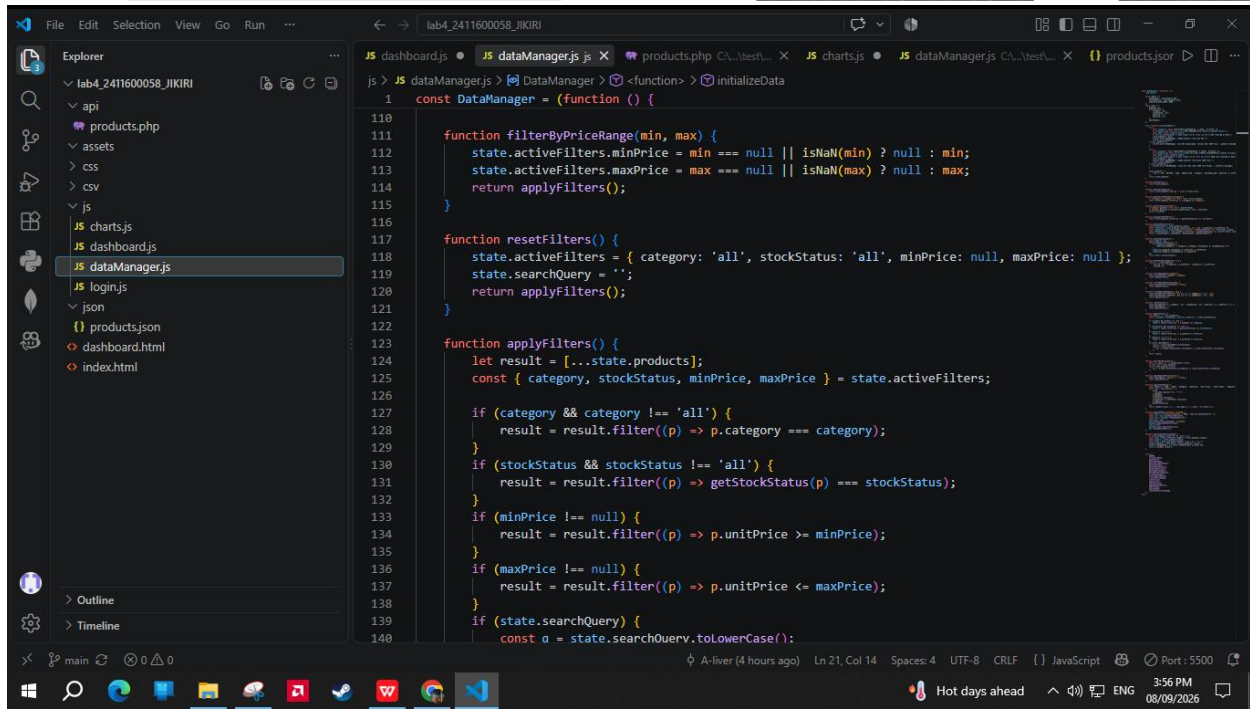
RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

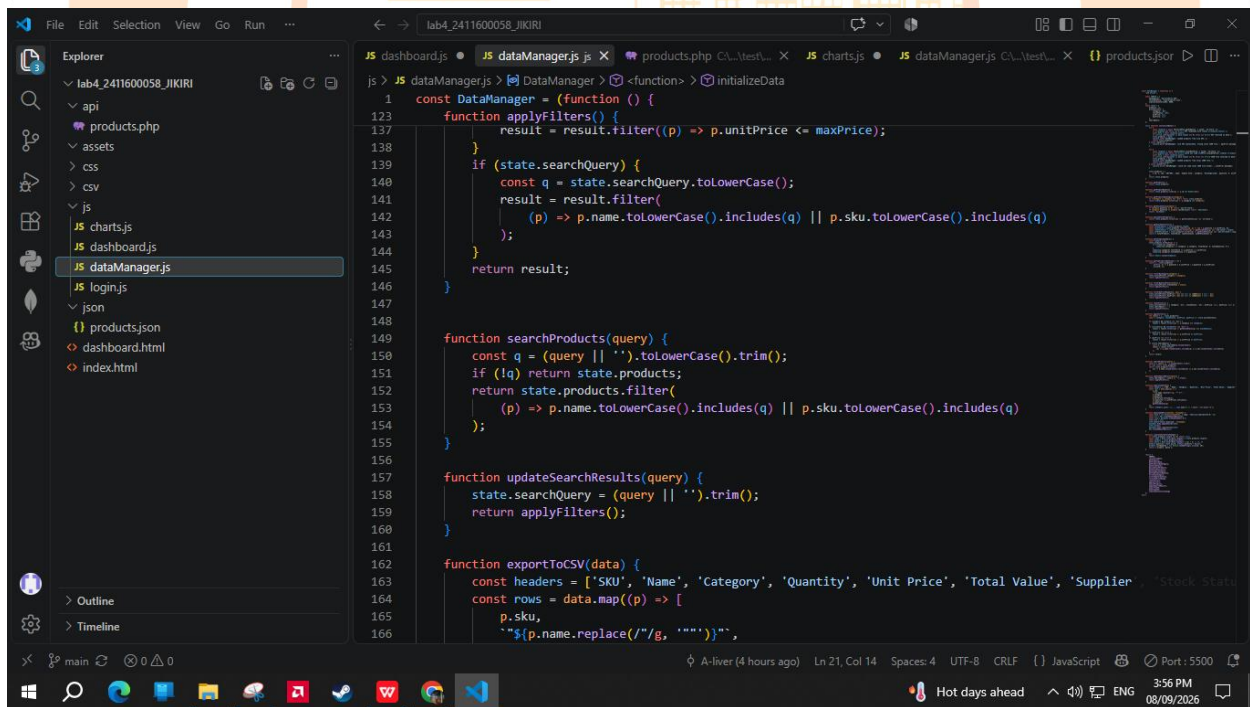
1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



```
1  const DataManager = (function () {  
110  
111      function filterByPriceRange(min, max) {  
112          state.activeFilters.minPrice = min === null || isNaN(min) ? null : min;  
113          state.activeFilters.maxPrice = max === null || isNaN(max) ? null : max;  
114          return applyFilters();  
115      }  
116  
117      function resetFilters() {  
118          state.activeFilters = { category: 'all', stockStatus: 'all', minPrice: null, maxPrice: null };  
119          state.searchQuery = '';  
120          return applyFilters();  
121      }  
122  
123      function applyFilters() {  
124          let result = [...state.products];  
125          const { category, stockStatus, minPrice, maxPrice } = state.activeFilters;  
126  
127          if (category && category !== 'all') {  
128              result = result.filter((p) => p.category === category);  
129          }  
130          if (stockStatus && stockStatus !== 'all') {  
131              result = result.filter((p) => getStockStatus(p) === stockStatus);  
132          }  
133          if (minPrice !== null) {  
134              result = result.filter((p) => p.unitPrice >= minPrice);  
135          }  
136          if (maxPrice !== null) {  
137              result = result.filter((p) => p.unitPrice <= maxPrice);  
138          }  
139          if (state.searchQuery) {  
140              const q = state.searchQuery.toLowerCase();  
141              result = result.filter((p) => p.name.toLowerCase().includes(q) || p.sku.toLowerCase().includes(q));  
142          }  
143          return result;  
144      }  
145  
146      function searchProducts(query) {  
147          const q = (query || '').toLowerCase().trim();  
148          if (!q) return state.products;  
149          return state.products.filter((p) => p.name.toLowerCase().includes(q) || p.sku.toLowerCase().includes(q));  
150      }  
151  
152      function updateSearchResults(query) {  
153          state.searchQuery = (query || '').trim();  
154          return applyFilters();  
155      }  
156  
157      function exportToCsv(data) {  
158          const headers = ['SKU', 'Name', 'Category', 'Quantity', 'Unit Price', 'Total Value', 'Supplier', 'Stock Status'];  
159          const rows = data.map((p) => {  
160              p.sku,  
161              `${p.name.replace(/"/g, '""')}`,  
162              p.category,  
163              p.quantity,  
164              p.unitPrice,  
165              p.unitPrice * p.quantity,  
166              p.supplier,  
167              p.stockStatus  
168          });  
169          const csvContent = headers.map(h => `"${h}"`).join(',') + '\n' + rows.map(r => r.map(c => `"${c}"`).join(',') + '\n').join('');  
170          const blob = new Blob([csvContent], { type: 'text/csv' });  
171          const url = window.URL.createObjectURL(blob);  
172          const a = document.createElement('a');  
173          a.href = url;  
174          a.download = 'products.csv';  
175          a.click();  
176          window.URL.revokeObjectURL(url);  
177      }  
178  
179      return { filterByPriceRange, resetFilters, applyFilters, searchProducts, updateSearchResults, exportToCsv };  
180  })();
```



```
1  const DataManager = (function () {  
123      function applyFilters() {  
124          result = result.filter((p) => p.unitPrice <= maxPrice);  
125      }  
126      if (state.searchQuery) {  
127          const q = state.searchQuery.toLowerCase();  
128          result = result.filter((p) => p.name.toLowerCase().includes(q) || p.sku.toLowerCase().includes(q));  
129      }  
130      return result;  
131  }  
132  
133  function searchProducts(query) {  
134      const q = (query || '').toLowerCase().trim();  
135      if (!q) return state.products;  
136      return state.products.filter((p) => p.name.toLowerCase().includes(q) || p.sku.toLowerCase().includes(q));  
137  }  
138  
139  function updateSearchResults(query) {  
140      state.searchQuery = (query || '').trim();  
141      return applyFilters();  
142  }  
143  
144  function exportToCsv(data) {  
145      const headers = ['SKU', 'Name', 'Category', 'Quantity', 'Unit Price', 'Total Value', 'Supplier', 'Stock Status'];  
146      const rows = data.map((p) => {  
147          p.sku,  
148          `${p.name.replace(/"/g, '""')}`,  
149          p.category,  
150          p.quantity,  
151          p.unitPrice,  
152          p.unitPrice * p.quantity,  
153          p.supplier,  
154          p.stockStatus  
155      });  
156      const csvContent = headers.map(h => `"${h}"`).join(',') + '\n' + rows.map(r => r.map(c => `"${c}"`).join(',') + '\n').join('');  
157      const blob = new Blob([csvContent], { type: 'text/csv' });  
158      const url = window.URL.createObjectURL(blob);  
159      const a = document.createElement('a');  
160      a.href = url;  
161      a.download = 'products.csv';  
162      a.click();  
163      window.URL.revokeObjectURL(url);  
164  }  
165  
166  return { filterByPriceRange, resetFilters, applyFilters, searchProducts, updateSearchResults, exportToCsv };  
167  })();
```



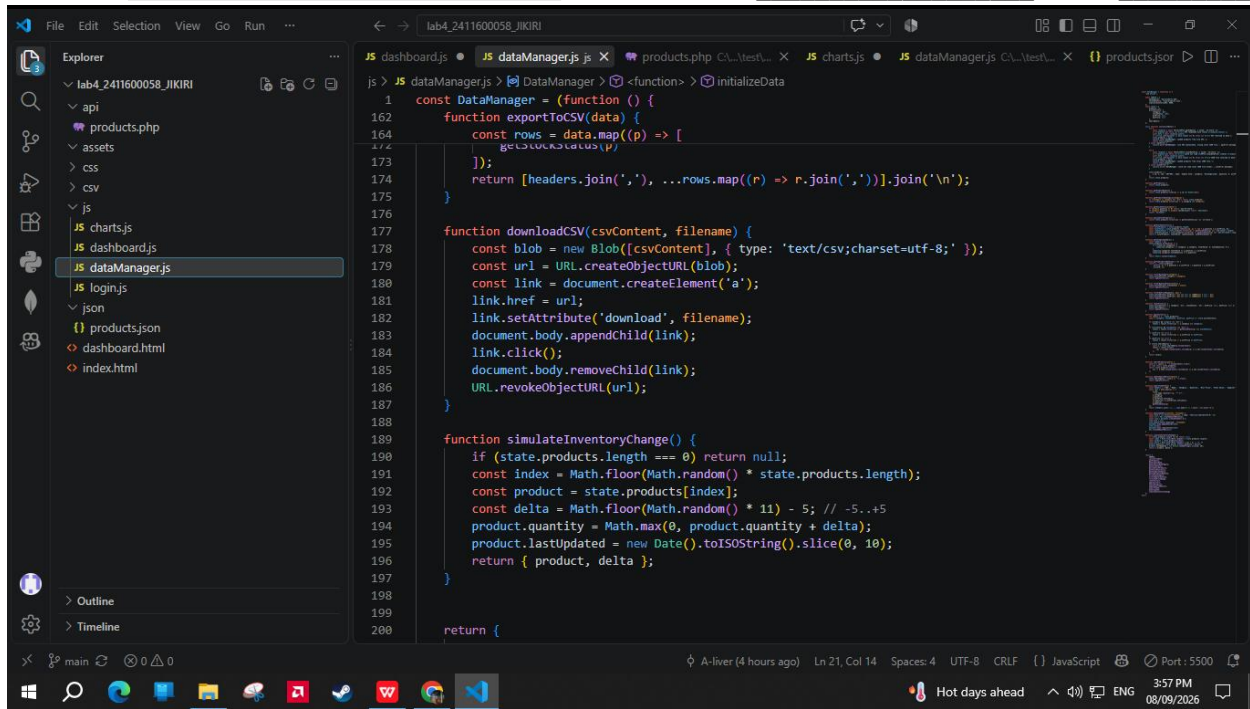

RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

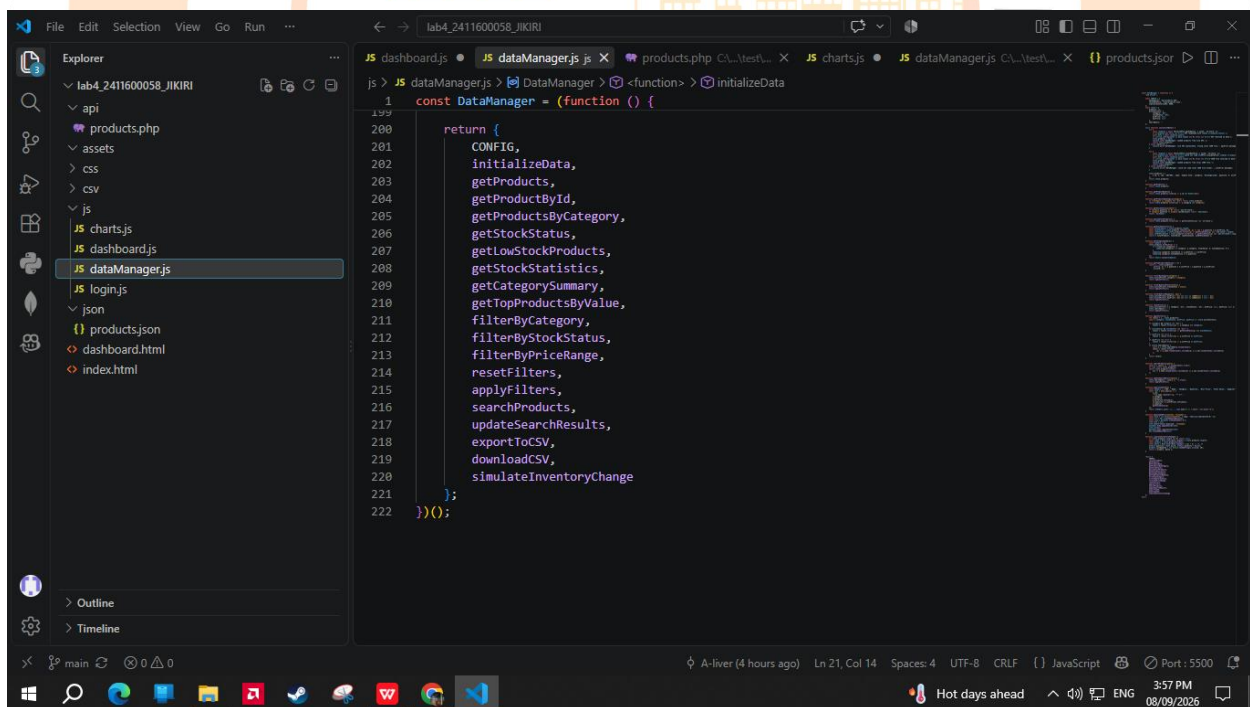
1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____



```
1  const DataManager = (function () {
162      function exportToCSV(data) {
164          const rows = data.map((p) => {
174              // ...
175          });
176          return [headers.join(','), ...rows.map((r) => r.join(',')].join('\n');
177      }
178
179      function downloadCSV(csvContent, filename) {
180          const blob = new Blob([csvContent], { type: 'text/csv;charset=utf-8;' });
181          const url = URL.createObjectURL(blob);
182          const link = document.createElement('a');
183          link.href = url;
184          link.setAttribute('download', filename);
185          document.body.appendChild(link);
186          link.click();
187          document.body.removeChild(link);
188          URL.revokeObjectURL(url);
189      }
190
191      function simulateInventoryChange() {
192          if (state.products.length === 0) return null;
193          const index = Math.floor(Math.random() * state.products.length);
194          const product = state.products[index];
195          const delta = Math.floor(Math.random() * 11) - 5; // -5..+5
196          product.quantity = Math.max(0, product.quantity + delta);
197          product.lastUpdated = new Date().toISOString().slice(0, 10);
198          return { product, delta };
199      }
200      return {
201          // ...
202      };
203  })();
```



```
1  const DataManager = (function () {
199      // ...
200      return {
201          CONFIG,
202          initializeData,
203          getProducts,
204          getProductById,
205          getProductsByCategory,
206          getStockStatus,
207          getLowStockProducts,
208          getStockStatistics,
209          getCategorySummary,
210          getTopProductsByValue,
211          filterByCategory,
212          filterByStockStatus,
213          filterByPriceRange,
214          resetFilters,
215          applyFilters,
216          searchProducts,
217          updateSearchResults,
218          exportToCSV,
219          downloadCSV,
220          simulateInventoryChange
221      };
222  })();
```



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
1 let categoryChart = null;
2 let stockStatusChart = null;
3 let topProductsChart = null;
4
5 const THEME_COLORS = ['#606C38', '#8C6C25', '#DDA15E', '#283618', '#8a9a5b', '#a3b18a',
6
7
8 function renderCharts() {
9   renderCategoryChart(DataManager.getCategorySummary());
10  renderStockStatusChart(DataManager.getStockStatistics());
11  renderTopProductsChart(DataManager.getTopProductsByValue(10));
12 }
13
14 function renderCategoryChart(summary) {
15   const canvas = document.getElementById('categoryValueChart');
16   if (!canvas || typeof Chart === 'undefined') return;
17
18   if (categoryChart) categoryChart.destroy();
19
20   categoryChart = new Chart(canvas, {
21     type: 'bar',
22     data: {
23       labels: summary.map(s => s.category),
24       datasets: [{
25         label: 'Inventory Value ($)',
26         data: summary.map(s => Number(s.totalValue.toFixed(2))),
27         backgroundColor: THEME_COLORS
28       }]
29     },
30     options: {
31       responsive: true,
32       maintainAspectRatio: false,
```

```
14 function renderCategoryChart(summary) {
39   });
40 }
41
42 function renderStockStatusChart(stats) {
43   const canvas = document.getElementById('stockStatusChart');
44   if (!canvas || typeof Chart === 'undefined') return;
45
46   const inStock = Math.max(0, stats.totalProducts - stats.lowStockCount - stats.outOfStockCount);
47
48   if (stockStatusChart) stockStatusChart.destroy();
49
50   stockStatusChart = new Chart(canvas, {
51     type: 'doughnut',
52     data: {
53       labels: ['In Stock', 'Low Stock', 'Out of Stock'],
54       datasets: [{
55         data: [inStock, stats.lowStockCount, stats.outOfStockCount],
56         backgroundColor: ['#28a745', '#ffc107', '#dc3545']
57       }]
58     },
59     options: {
60       responsive: true,
61       maintainAspectRatio: false,
62       plugins: {
63         title: { display: true, text: 'Stock Status Distribution' }
64       }
65     }
66   });
67 }
68
69 function renderTopProductsChart(products) {
```



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
67 }
68
69 function renderTopProductsChart(products) {
70   const canvas = document.getElementById('topProductsChart');
71   if (!canvas || typeof Chart === 'undefined') return;
72
73   if (topProductsChart) topProductsChart.destroy();
74
75   topProductsChart = new Chart(canvas, {
76     type: 'bar',
77     data: {
78       labels: products.map(p => p.name),
79       datasets: [
80         {
81           label: 'Value ($)',
82           data: products.map(p => Number((p.quantity * p.unitPrice).toFixed(2))),
83           backgroundColor: '#8C6C25'
84         }
85       ],
86     },
87     options: {
88       indexAxis: 'y',
89       responsive: true,
90       maintainAspectRatio: false,
91       plugins: {
92         legend: { display: false },
93         title: { display: true, text: 'Top 10 Products by Value' }
94       }
95     }
96   });
```

```
143 }
144
145 .chart-container {
146   position: relative;
147   height: 300px;
148 }
149 @media (max-width: 768px) {
150   .sidebar {
151     min-height: auto !important;
152     max-height: auto !important;
153   }
154
155   .stat-card .card-text {
156     font-size: 1.4rem;
157   }
158
159   .chart-container {
160     height: 220px;
161   }
162
163   .filter-card .btn-group {
164     flex-wrap: wrap;
165   }
166
167   .filter-card .btn-group .btn {
168     flex: 1 1 25%;
169     font-size: 0.8rem;
170   }
171 }
172
173 /* Custom Scrollbar */
```




RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
css > # style.css > .chart-container
4200
201
202
203 /* Chart cards */
204 .chart-card {
205     border: none;
206     border-radius: 12px;
207     box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
208     transition: box-shadow 0.3s ease;
209 }
210
211 .chart-card:hover {
212     box-shadow: 0 6px 20px rgba(0, 0, 0, 0.12);
213 }
214
215 /* Filter & search panel */
216 .filter-card {
217     border: none;
218     border-radius: 12px;
219     background-color: #ffffff;
220     box-shadow: 0 2px 10px rgba(0, 0, 0, 0.06);
221 }
222
223 .filter-card .btn-group .btn.active {
224     background-color: var(--theme-primary);
225     border-color: var(--theme-primary);
226     color: #ffffff;
227 }
228
229 .filter-card .form-control:focus,
230 .filter-card .form-select:focus {
231     border-color: var(--theme-primary);
```

```
css > # style.css > .chart-container
4229
230 .filter-card .form-select:focus {
231     border-color: var(--theme-primary);
232     box-shadow: 0 0 0.2rem rgba(96, 188, 56, 0.25);
233 }
234
235 /* Inventory table row states */
236 #inventoryTableBody tr {
237     transition: background-color 0.3s ease;
238 }
239
240 .row-low-stock {
241     background-color: rgba(243, 167, 18, 0.12) !important;
242 }
243
244 .row-out-of-stock {
245     background-color: rgba(220, 53, 69, 0.1) !important;
246 }
247
248 .row-out-of-stock td {
249     color: #842029;
250 }
251
252 /* Search highlight */
253 #inventoryTableBody mark {
254     background-color: var(--theme-accent);
255     color: #000000;
256     padding: 0 2px;
257     border-radius: 3px;
258 }
259
260 /* Low stock alert banner */
```



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

```
css > # style.css > .chart-container
258
259
260 /* Low stock alert banner */
261 #lowStockAlert .alert {
262   border-left: 5px solid var(--bs-warning);
263   border-radius: 10px;
264   animation: alertSlideIn 0.4s ease;
265 }
266
267 #lowStockAlert ul {
268   padding-left: 1.1rem;
269 }
270
271 @keyframes alertSlideIn {
272   from {
273     opacity: 0;
274     transform: translateY(-8px);
275   }
276   to {
277     opacity: 1;
278     transform: translateY(0);
279   }
280 }
281
282 /* Export button */
283 #exportCsvBtn {
284   transition: transform 0.15s ease;
285 }
286
287 #exportCsvBtn:hover {
288   transform: translateY(-2px);
289 }
```

```
css > # style.css > .chart-container
285
286
287 #exportCsvBtn:hover {
288   transform: translateY(-2px);
289 }
290
291 /* Toast notifications for real-time simulation */
292 #toastContainer .toast {
293   border-radius: 10px;
294 }
295
296 /* Loading spinner container */
297 #inventoryLoading .spinner-border {
298   width: 3rem;
299   height: 3rem;
300 }
301
302 /* Responsive tweaks for charts/filters on small screens */
303 @media (max-width: 768px) {
304   .chart-container {
305     height: 220px;
306   }
307
308   .filter-card .btn-group {
309     flex-wrap: wrap;
310   }
311
312   .filter-card .btn-group .btn {
313     flex: 1 1 25%;
314     font-size: 0.8rem;
315   }
316 }
```



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

restaurant Management Welcome, admin [Logout](#)

Dashboard

Profile

Settings

Logout

Good Evening, admin!

TODAY'S ORDERS
32

TODAY'S REVENUE
\$19,150

RESERVATION
12

AVAILABLE TABLES
8

Inventory Management

9 item(s) need attention

- Lamb Chops — OUT OF STOCK
- Bell Peppers — OUT OF STOCK
- Black Pepper (kg) — 3 left (reorder at 5)
- Coffee Beans (kg) — 6 left (reorder at 8)
- Shrimp — 8 left (reorder at 10)
- Baguette — 9 left (reorder at 10)

TOTAL PRODUCTS
23

INVENTORY VALUE
\$2,428.40

LOW STOCK
7

OUT OF STOCK
2

TOTAL PRODUCTS
23

INVENTORY VALUE
\$2,428.40

LOW STOCK
7

OUT OF STOCK
2

Inventory Value by Category

Category	In Stock (\$)	Low Stock (\$)	Out of Stock (\$)
Meat & Poultry	200	100	0
Seafood	400	100	0
Vegetables & Produce	100	100	0
Dairy & Eggs	200	100	0
Beverages	400	100	0
Dry Goods & Spices	100	100	0
Bakery & Bread	100	100	0

Stock Status Distribution

Stock Status	Count
In Stock	18
Low Stock	7
Out of Stock	2

Top 10 Products by Value

Product	Value (\$)
Tuna Steak	350
Soft Drinks (case)	300
Bottled Water (case)	250
Salmon Fillet	200
Beef Tenderloin	150
Ground Beef	100
Chicken Breast	100
Eggs (dozen)	100
Flour	100
Shrimp	100

Category

All Categories

Stock Status

All In Low Out

Min Price

0.00

Max Price

0.00

Apply

Reset

Search by name or SKU

Tuna Steak: +4 units -- now 26



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

127.0.0.1:5500/dashboard.html

Category: All Categories | Stock Status: All In Low Out | Min Price: 0.00 | Max Price: 0.00 | Apply | Reset

Search by name or SKU: lamb | Export to CSV

SKU	Product	Category	Qty	Unit Price	Total Value	Status	Supplier	Last Updated
INV-003	Lamb Chops	Meat & Poultry	0	\$6.80	\$0.00	Out of Stock	FreshFarms Co.	2026-09-03

Recent Activity

Date	Activity
2026-08-10 14:30	New reservation received for Table 8

Bread Rolls (dozen): +2 units → now 22

Save As

File name: inventory_export_2026-09-08

Save as type: XLS Worksheet

Save | Cancel



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

WPS Office Laboratory Exercise 4 ENHANCIN inventory_export_2026-09-08

File Home Insert Page Layout Formulas Data Review View Tools Premium WPS AI Not uploaded Share

Calibri 11 B I U ab Wrap Text General Format Conversion Conditional Formatting Editing

A1 fx SKU

SKU	Name	Category	Quantity	Unit Price	Total Value	Supplier	Stock Status
INV-003	Lamb Chop	Meat & Pou	0	6.8	0	FreshFarms	out-of-stock

inventory_export_2026-09-08

32°C Partly sunny 4:05 PM 08/09/2026

localhost/lab1_2411600058_Jikiri/lab4_2411600058_Jikiri/dashboard.html

Dimensions: Responsiw 760 x 906 Fit to window No throttling Save-Data: default

Inventory Management

9 items need attention

- Lamb Chops - OUT OF STOCK
- Bell Peppers - OUT OF STOCK
- Black Pepper (kg) - 8 left (reorder at 10)
- Coffee Beans (kg) - 6 left (reorder at 8)
- Shrimp - 8 left (reorder at 10)
- Baguettes - 9 left (reorder at 10)

TOTAL PRODUCTS: 23 INVENTORY VALUE: \$2328.00

LOW STOCK: 7 OUT OF STOCK: 2

Inventory Value by Category

Stock Status Distribution

products.php 200 fetch dataManager.js:22 4.4 kB 24 ms

1 / 13 requests 4.4 kB / 5.0 kB transferred 4.1 kB / 574 kB resources Finish: 542 ms DOMContentLoaded

33°C Mostly cloudy 5:27 PM 08/09/2026



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

github.com/A-liver/lab1_2411600058_jikiri/tree/main/lab4_2411600058_JIKIRI

A-liver / lab1_2411600058_jikiri

Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings

main lab1_2411600058_jikiri / lab4_2411600058_JIKIRI

A-liver Lab 4 completed b1f9704 · 4 hours ago History

Name	Last commit message	Last commit date
..		
api	Lab 4 completed	4 hours ago
css	Lab 4 completed	4 hours ago
js	Lab 4 completed	4 hours ago
json	added some data	yesterday
dashboard.html	Lab 4 completed	4 hours ago
index.html	Adding Some Files	yesterday

github.com/A-liver/lab1_2411600058_jikiri/commits/main/lab4_2411600058_JIKIRI

A-liver / lab1_2411600058_jikiri

Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings

Commits

History for lab1_2411600058_jikiri / lab4_2411600058_JIKIRI on main

Commits on Sep 8, 2026

- Lab 4 completed
A-liver committed 4 hours ago b1f9704
- separated the dashboard and charts
A-liver committed 5 hours ago 8bc9f53
- make the charts visible and it uses the JSON data
A-liver committed 15 hours ago 01dd6ca

Commits on Sep 7, 2026

- added some data
A-liver committed yesterday bb95d47
- Adding Some Files
A-liver committed yesterday ec9ad79

End of commit history for this file



RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

Conclusion

In this laboratory exercise I transformed my static dashboard from the previous laboratory exercise, making it an interactive inventory management system using and adding Chart.js for the visualizations and adding filtering, search, stock alerts, CSV export into it and connecting it with an API php. Making the webpage also helped me understand the importance of using real backend for persistent, shared inventory data since Live Server can't execute PHP. Overall I learned how DOM manipulation, event-driven updates, third-party libraries and asynchronous data fetching helps turn a static webpage into a functional interactive one.





RAMON MAGSAYSAY MEMORIAL COLLEGES

Information Technology Education Program

1st SEMESTER: AY: 2026 - 2027



NAME: _____ SCHEDULE: _____ SCORE: _____
SUBJECT: **WEB SYSTEMS AND TECHNOLOGIES** INSTRUCTOR: _____ DATE: _____

